

Apostila de Revisão

Lógica de Programação

1. O que é Lógica de Programação

Lógica de programação é a habilidade de **pensar de forma organizada, sequencial e sem ambiguidades**, transformando um problema em passos claros que podem ser executados por um computador.

Programar não começa no código. Programar começa no **raciocínio**.

Antes de existir qualquer linguagem (Java, Python, JavaScript, C, etc.), existe a lógica. A linguagem apenas traduz essa lógica para uma forma que a máquina entende.

Exemplo

Imagine que você precise explicar para alguém como ligar um computador:

1. Apertar o botão de energia
2. Aguardar o sistema iniciar
3. Digitar a senha
4. Acessar a área de trabalho

Se você pular um passo, o processo falha. O computador funciona da mesma forma: **ele só faz exatamente o que foi mandado**.

2. Algoritmo: a base de tudo

Um **algoritmo** é uma sequência lógica, finita e ordenada de passos que descrevem como resolver um problema.

Todo algoritmo precisa responder três perguntas:

- O que entra? (dados de entrada)
- O que acontece? (processamento)
- O que sai? (resultado)

Exemplo

Pense em um caixa eletrônico:

- Entrada: valor solicitado pelo cliente
- Processamento: verificar saldo e limites
- Saída: liberar dinheiro ou mostrar mensagem de erro

Mesmo sistemas complexos são apenas **algoritmos bem organizados**.

3. Linguagem estruturada

A lógica de programação inicial trabalha com o paradigma **estruturado**, onde o código é executado:

- De cima para baixo
- Linha por linha
- Com desvios controlados por decisões

Esse modelo é essencial para:

- Entender fluxo de execução
- Evitar erros lógicos
- Criar programas previsíveis

Aprender lógica estruturada antes de orientação a objetos **não é opção**, é requisito.

4. Variáveis: memória controlada

Uma **variável** é um espaço reservado na memória do computador para armazenar informações temporárias durante a execução de um programa.

Ela funciona como uma caixa identificada, onde:

- O nome identifica a caixa
- O tipo define o que pode ser guardado
- O valor é o conteúdo atual

Exemplo

Considere um sistema de cadastro:

- Nome do cliente
- Idade do cliente
- Saldo disponível

Cada uma dessas informações precisa de uma variável diferente, pois possuem naturezas diferentes.

Misturar tipos causa erro lógico.

5. Tipos de dados fundamentais

Os tipos de dados definem **como o computador interpreta e armazena a informação**.

Inteiro

Utilizado para contagens, quantidades e identificadores numéricos.

Exemplos práticos:

- Quantidade de alunos
- Número de produtos
- Idade de uma pessoa

Real

Utilizado quando há necessidade de precisão decimal.

Exemplos práticos:

- Preço de um produto
- Salário
- Medidas

Caractere

Utilizado para textos e símbolos.

Exemplos práticos:

- Nome de usuário
- CPF (não é número matemático)
- Senha

Lógico

Utilizado para decisões.

Exemplos práticos:

- Usuário autenticado? (verdadeiro ou falso)
- Produto disponível?

6. Declaração e atribuição

Declaração

Declarar uma variável significa informar ao sistema que ela existe e qual tipo de dado irá armazenar.

Sem declaração, o computador não sabe onde guardar a informação.

Atribuição

Atribuir é colocar um valor dentro da variável.

Exemplo

Imagine um formulário:

- Você cria o campo "idade" (declaração)
- O usuário digita 30 (atribuição)

Esses dois momentos são diferentes e sempre aparecem na programação.

7. Entrada e saída de dados

A partir deste ponto, iniciamos oficialmente o **uso do Visualg**, aplicando os conceitos de lógica em pseudocódigo executável.

Conceito

Entrada de dados ocorre quando o programa **recebe informações externas**, normalmente digitadas pelo usuário. Saída de dados ocorre quando o programa **apresenta informações processadas**.

Nenhum sistema real funciona sem entrada e saída. Um programa que apenas calcula internamente, sem interagir, tem uso extremamente limitado.

Todo programa segue o ciclo fundamental:

Entrada → Processamento → Saída

Exemplo – Boas-vindas ao usuário

Contexto: Um sistema simples de atendimento precisa solicitar o nome do usuário e exibir uma mensagem personalizada.

Algoritmo "BoasVindas"

Var

nome : caractere

Inicio

escreva("Digite seu nome: ")

leia(nome)

escreva("Bem-vindo(a), ", nome)

Fimalgoritmo

Explicação passo a passo:

- O algoritmo é iniciado com um nome identificador
- A variável nome é declarada para armazenar texto
- escreva exibe uma mensagem na tela

- leia aguarda o usuário digitar
- O valor digitado é reutilizado na mensagem final

Prática – Entrada e saída

Proposta: Crie um algoritmo que solicite a idade do usuário e mostre a mensagem:

"Sua idade é X anos"

8. Operadores aritméticos

Operadores aritméticos são utilizados para realizar cálculos matemáticos dentro do programa.

Sem eles, o sistema apenas recebe e mostra dados, mas não gera informação útil.

Exemplo – Soma de dois números

Contexto: Um sistema precisa calcular a soma de dois valores informados pelo usuário.

Algoritmo "SomaNumeros"

Var

n1, n2, soma : real

Inicio

escreva("Digite o primeiro número: ")

leia(n1)

escreva("Digite o segundo número: ")

leia(n2)

soma <- n1 + n2

escreva("Resultado da soma: ", soma)

Fimalgoritmo

Explicação:

- Os números são lidos separadamente
- O operador + soma os valores
- O resultado é armazenado em outra variável
- O valor final é exibido

Prática – Operações aritméticas

Proposta: Crie um algoritmo que leia dois números e mostre o resultado da multiplicação.

9. Ordem de precedência

A ordem de precedência define **qual operação será executada primeiro**.

Exemplo

Considere o cálculo:

$$2 + 5 \times 2$$

Se a multiplicação não for feita primeiro, o resultado será incorreto.

Por isso, o uso de parênteses é uma ferramenta lógica para evitar erros e deixar o código claro.

10. Operadores relacionais

Operadores relacionais são utilizados para **comparar valores**.

Exemplo – Verificação de idade

Contexto: Um sistema deve verificar se uma pessoa é maior de idade.

Algoritmo "Verificaldade"

Var

idade : inteiro

Inicio

```
escreva("Digite sua idade: ")
```

```
leia(idade)
```

```
escreva(idade >= 18)
```

Fimalgoritmo

O resultado exibido será **verdadeiro** ou **falso**, mostrando como comparações funcionam.

11. Operadores lógicos

Operadores lógicos permitem combinar condições.

Exemplo

Para liberar um acesso:

- Usuário está cadastrado
- Senha está correta

Ambas precisam ser verdadeiras ao mesmo tempo.

Eles tornam as decisões mais completas e realistas.

12. Estruturas de decisão

Estruturas de decisão permitem que o programa **tome decisões automaticamente** com base em condições.

Sem estruturas de decisão, todo programa executaria sempre o mesmo caminho, o que não representa sistemas reais.

Exemplo – Verificação de maioridade

Contexto: Um sistema deve permitir ou negar acesso com base na idade.

Algoritmo "Maioridade"

Var

idade : inteiro

Inicio

```
escreva("Digite sua idade: ")
```



```
leia(idade)

se idade >= 18 entao
    escreva("Acesso permitido")
senao
    escreva("Acesso negado")
fimse

Fimalgoritmo
```

Explicação:

- A condição compara a idade com 18
- Se for verdadeira, executa o bloco entao
- Caso contrário, executa o bloco senao

Prática – Estrutura SE

Proposta: Crie um algoritmo que leia um número e informe se ele é positivo ou negativo.

13. Introdução às repetições

Estruturas de repetição são usadas quando uma ação precisa ser executada várias vezes automaticamente.

Exemplo explicado – Contador de 1 a 5

Contexto: Um sistema precisa exibir números em sequência.

Algoritmo "Contador"

Var

c : inteiro

Inicio

c <- 1

enquanto c <= 5 faca

escreva(c)

c <- c + 1

fimenquanto

Fimalgoritmo

Explicação:

- A variável c inicia em 1
- Enquanto a condição for verdadeira, o bloco executa
- O contador é incrementado

Prática – Estrutura ENQUANTO

Proposta: Crie um algoritmo que mostre os números de 1 até 10.

Encerramento

Esta apostila não ensina a programar.

Ela ensina a **pensar como programador**.

Quando a prática começar, o raciocínio já estará formado e o código será apenas a formalização desse pensamento.